Banking News and Deal Intelligence Engine: Architectural Blueprint and Implementation Guide (Feb 2025 – May 2025)

Executive Summary

The financial markets of early 2025, specifically the Foreign Exchange (FX), Credit, and Rates sectors, operate at a velocity that defies manual analysis. For institutional desks, the latency between a news event—such as a central bank governor's nuanced comment on yield curve control or a subtle shift in credit spreads for a Global Systemically Important Bank (G-SIB)—and market reaction is measured in milliseconds. The requested Banking News and Deal Intelligence Engine represents a critical infrastructure project designed to bridge the gap between massive, unstructured information flows and actionable deal intelligence. By leveraging a Python-centric stack integrating **BeautifulSoup** for ingestion, **spaCy** for granular entity extraction, **BERTopic** for semantic clustering, and transformer-based summarization, this system aims to autonomously process over 100 daily articles to produce high-signal morning summaries.

This report serves as an exhaustive, step-by-step implementation guide for building this engine from scratch. It addresses the unique challenges of the February 2025 to May 2025 window, a period anticipated to be characterized by heightened volatility in sovereign debt markets and shifting credit cycles. The analysis moves beyond simple code execution to explore the theoretical and practical application of Natural Language Processing (NLP) in finance, detailing how to extract "deal" information—identifying issuers, maturities, and currencies—and track regulatory shifts that could precipitate market-wide contagion.

1. Strategic Architecture and System Design

The development of a financial intelligence engine is not merely a software engineering task; it is an exercise in information arbitrage. The goal is to capture, structure, and synthesize information faster and more accurately than the consensus. The architecture proposed herein is modular, scalable, and built entirely on open-source Python libraries, ensuring flexibility and cost-effectiveness while maintaining enterprise-grade reliability.

1.1 The Operational Context: FX, Credit, and Rates Markets

The target domains—FX, Credit, and Rates—possess distinct linguistic and structural characteristics that dictate the engineering approach.

- **FX Markets:** Driven by macroeconomic data and central bank rhetoric. News is often

continuous, repetitive, and heavily heavily reliant on specific currency pairs (e.g., EUR/USD, GBP/JPY).

- **Rates Markets:** Focus on sovereign bonds (Treasuries, Bunds, JGBs). The terminology is highly technical, involving "tenors" (maturities), "basis points," and "yield curves."
- **Credit Markets:** Centered on corporate issuers and deal flow. Information here is episodic—new issuance announcements, rating changes, or default warnings.[1]

The system must be capable of distinguishing between a "10-year high" (a statistical observation) and a "10-year Note" (a tradable instrument). This semantic disambiguation is the core challenge that the NLP pipeline must solve.[2]

1.2 High-Level Pipeline Architecture

The system follows a classic Extract, Transform, Load (ETL) paradigm, enhanced with advanced NLP components. This "ELT-NLP" pipeline ensures that raw data is preserved for future model training while processed insights are delivered in near real-time.

Stage	Function	Core Technologies
Ingestion	Data Harvesting	Playwright, BeautifulSoup, Requests, External APIs
Preprocessing	Cleaning & Normalization	Regex, Pandas, NLTK
Extraction (NER)	Entity Identification	spaCy (Transformers + EntityRuler)
Clustering	Topic Modeling	BERTopic, UMAP, HDBSCAN, FinBERT
Summarization	Insight Generation	Hugging Face Transformers (Pegasus, BART)
Orchestration	Scheduling & Dependency	Apache Airflow, Docker
Reporting	Delivery	Jinja2, SMTP, HTML

The chosen stack—Python—is the industry standard for financial data science due to its rich ecosystem.[3] The project structure should adhere to the "Cookiecutter Data Science" methodology to ensure reproducibility and modularity, separating data, source code, and configuration files.[4]

---

2. Data Acquisition: Building the Feed Handler

The lifeblood of the intelligence engine is the quality of its input data. Relying on a single source creates a single point of failure and a biased view of the market. Therefore, the acquisition layer is designed as a hybrid system that ingests structured data from APIs and unstructured narrative data via web scraping.

2.1 Structured Data Integration: The API Layer

While the primary goal is analyzing news, quantitative context is essential. An article stating

"Euro plummets" needs to be contextualized with the actual exchange rate to determine the severity of the move.

2.1.1 Alpha Vantage and Financial Modeling Prep (FMP)

For the FX and Rates data, **Alpha Vantage** provides a robust set of APIs. The system requires real-time and historical data for major currency pairs and economic indicators.[6] Similarly, **Financial Modeling Prep (FMP)** offers a specialized "Stock News" endpoint and forex data.[7]

- **Implementation Strategy:** The engine initiates a daily poll of these APIs at 06:00 AM local time (pre-market). It requests the Open-High-Low-Close (OHLC) data for the target currencies (EUR, GBP, JPY, AUD) and major government bond yields. This data is stored in a local SQLite or PostgreSQL database to serve as the "ground truth" for the day's analysis.

2.1.2 EODHD Financial News API

EODHD offers a specialized "Financial News API" that not only aggregates news but provides pre-calculated sentiment scores.[8] Integrating this allows the system to benchmark its internal sentiment analysis against a commercial provider. The endpoint https://eodhd.com/api/news can be queried with specific tickers (e.g., EURUSD.FOREX) to retrieve a curated feed, reducing the burden on the web scraper for highly liquid assets.

2.2 Unstructured Data Acquisition: The Web Scraper

Much of the nuance in credit and rates markets is found in specialized portals that do not offer public APIs. Constructing a robust, respectful, and resilient web scraper is the first major coding task.

2.2.1 Target Selection and Challenges

- **Investing.com:** A primary source for global economic calendars and bond market news. It is heavily dynamic, using JavaScript to inject content after the initial page load.[9]
- **ForexLive:** A critical source for real-time trader sentiment and central bank analysis. It offers RSS feeds, which are machine-friendly, but deep analysis requires scraping the full articles linked in the feed.[10]
- **Reuters/Bloomberg:** These sites have strict anti-bot measures. The scraper must be designed to handle "soft" blocks and render JavaScript effectively.[12]

2.2.2 The Technical Stack: Playwright vs. Selenium vs. BeautifulSoup

For this engine, a hybrid approach is recommended. **BeautifulSoup** is lightweight and fast but cannot execute JavaScript. **Selenium** is the traditional choice for dynamic sites but is resource-heavy and slower. **Playwright** is the modern alternative, offering faster execution, better support for modern web features (like Shadow DOM), and a robust headless mode.[13]

The Decision Matrix:

- **Static Content (RSS Feeds, Simple HTML):** Use Requests + BeautifulSoup. This covers ForexLive and basic news portals. It is fast and consumes minimal bandwidth.[15]

- **Dynamic Content (Investing.com, Yield Tables):** Use Playwright. It creates a browser context that can wait for network idle states, ensuring that tables and dynamic widgets are fully loaded before extraction.[16]

2.2.3 Implementing the Scraper Step-by-Step

The scraping module is built as a Python package within the src/scrapers directory.
Step 1: Session Management and Anti-Bot Strategies
Financial sites monitor for non-human behavior. The scraper must rotate User-Agents and implement intelligent delays.

Python

```
import time
import random
from playwright.sync_api import sync_playwright

def robust_request(url):
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        context = browser.new_context(
            user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36..."
        )
        page = context.new_page()
        page.goto(url, wait_until="networkidle")
        content = page.content()
        browser.close()
        return content
```

This snippet demonstrates the use of Playwright to bypass basic JavaScript checks.[13] The system should also implement randomized sleep intervals (e.g., time.sleep(random.uniform(2, 5))) between requests to mimic human reading patterns.[17]
Step 2: Content Extraction with BeautifulSoup
Once the raw HTML is retrieved, BeautifulSoup parses the DOM. The system targets specific semantic tags—typically <h1> for titles, div.article-content for the body, and <time> for publication timestamps.

- **Filtering:** To ensure relevance, the scraper applies a length filter. Text blocks shorter than 35 characters are often navigation links or captions and are discarded.[15]
- **Structure:** Extracted data is normalized into a dictionary format: { 'source': 'Investing.com', 'title': '...', 'body': '...', 'timestamp': '...', 'url': '...' }.

Step 3: Handling RSS Feeds
For sites like ForexLive, the scraper parses the RSS XML first to get a list of URLs. This is far

more efficient than crawling the homepage.

Python

```
import feedparser
d = feedparser.parse('https://www.forexlive.com/feed')
for entry in d.entries:
    process_article(entry.link)
```

This approach reduces the load on the target server and ensures the engine captures every new headline.[10]

---

3. Financial Natural Language Processing: The Preprocessing Layer

Raw text from financial news is notoriously messy. It is replete with tickers, percentages, currency symbols, and jargon that can confuse standard NLP models. Before entity extraction can begin, the text must be normalized.

3.1 Cleaning and Normalization Strategies

The src/utils/cleaning.py module handles text hygiene.

- **Ticker Normalization:** "EUR/USD", "EURUSD", and "Euro-Dollar" must be mapped to a single token: EURUSD. This unification is critical for the clustering algorithm to recognize that articles using different terminologies are discussing the same subject.
- **Date Normalization:** In credit markets, "maturity" is a key concept. Relative dates ("10-year note") must be distinguished from calendar dates ("the 2025 fiscal year"). The system uses Regex to identify and standardize bond tenors (e.g., converting "30yr", "30-year", and "30Y" to TENOR_30Y).[2]
- **Noise Removal:** Regex is used to strip boilerplate text such as "Read more," "Disclosure," and social media handles.[18]

3.2 Sentiment Analysis Preparation

While sophisticated sentiment analysis occurs later, the preprocessing stage tags words based on the **Loughran-McDonald Master Dictionary**. This lexicon is specifically tuned for financial text, recognizing that words like "liability" or "risk" have different connotations in finance than in general English.[20] The system adds a sentiment_score metadata field to each article during ingestion, providing a baseline metric for the "Morning Summary."

---

4. Advanced Entity Recognition: Extracting Deal

Intelligence

The core requirement of the engine is to extract "currencies, issuers, maturities, and regulatory events." Standard NLP models (like spaCy's default en_core_web_sm) are insufficient for this. They often mistake "Bond" (a name) for "bond" (the instrument) or fail to recognize complex bank names. The solution is a hybrid NER pipeline using **spaCy**.

4.1 The Hybrid NER Architecture

The pipeline combines statistical prediction (using en_core_web_trf, a transformer-based model) with rule-based matching (using spaCy's EntityRuler). This ensures high recall for known financial entities while retaining the ability to generalize.[22]

4.1.1 Implementing the EntityRuler

The EntityRuler allows us to define strict patterns that override or augment the statistical model. This is implemented in src/nlp/spacy_pipeline.py.

A. Currency Extraction:

Currencies in FX news almost always appear in pairs or with specific symbols.

- *Pattern:* [A-Z]{3}/[A-Z]{3} (e.g., EUR/USD).
- *Pattern:* [A-Z]{3}[A-Z]{3} (e.g., USDJPY).
- *Implementation:*
  Python
  ```
  patterns ={3}/?[A-Z]{3}$"}}]},
     {"label": "CURRENCY", "pattern": "Greenback"},  # Nickname for USD
     {"label": "CURRENCY", "pattern": "Loonie"},     # Nickname for CAD
  ]
  ```

  This ensures that colloquial market terms are captured as structured data.[24]

B. Maturity Extraction:

For the Rates market, the "tenor" is the defining attribute.

- *Pattern:* Regex is used to capture "2-year", "10Y", "30-yr".
- *Context Check:* A simple regex match is dangerous (it might capture "10-year anniversary"). The pipeline uses spaCy's dependency parser to check if the entity modifies a financial noun (Note, Bond, Yield, Treasury).
  Python
  ```
  # Pattern to catch '10-year', '5yr', etc.
  {"label": "MATURITY", "pattern":}
  ```

.[1]

C. Issuer Identification (G-SIBs):

Identifying the issuer is critical for Credit intelligence. The system utilizes the Financial Stability Board's list of Global Systemically Important Banks (G-SIBs) as a gazetteer.

- *Data Source:* The 2023/2024 G-SIB list includes entities like "JPMorgan Chase,"

"Deutsche Bank," "Sumitomo Mitsui," and "Société Générale".[26]

- *Implementation:* These names are loaded into the EntityRuler as ISSUER entities. This guarantees that any mention of a major bank is flagged, regardless of the context.
  Python
  issuers =
  for issuer in issuers:
  patterns.append({"label": "ISSUER", "pattern":})

4.2 Regulatory Event Detection

Regulatory news is a high-impact driver of market structure. The request specifically asks for the extraction of these events.

- *Taxonomy Construction:* The system uses a keyword taxonomy derived from compliance standards. Keywords include: "AML" (Anti-Money Laundering), "KYC" (Know Your Customer), "Basel III," "Sanctions," "SEC," "FCA," "Enforcement," "Fine," and "Penalty".[28]
- *Extraction Logic:* The pipeline flags an article as a REGULATORY_EVENT if it contains an intersection of a **Regulator** entity (e.g., "SEC") and a **Trigger** keyword (e.g., "fine").
  - *Example:* "The SEC fined Bank of America..." -> Regulator: SEC, Trigger: fined, Target: Bank of America.
  - *Insight:* This allows the morning summary to have a dedicated "Compliance & Regulation" section, alerting risk managers to potential contagion.[30]

---

5. Semantic Clustering with BERTopic

Analyzing 100+ articles daily results in significant redundancy. Major stories (e.g., a Federal Reserve rate decision) will be covered by every outlet. The engine must group these into a single "topic" to avoid repetitive summaries. **BERTopic** is the ideal tool for this, utilizing transformer embeddings to find dense clusters of semantically similar texts.[32]

5.1 The Clustering Algorithm

The BERTopic pipeline consists of several modular steps, each configured for financial text.

5.1.1 Embeddings: The FinBERT Advantage

The quality of clustering depends on the quality of the vector embeddings. Standard models (like all-MiniLM-L6-v2) are trained on general text. For this engine, we use **FinBERT** (or sentence-transformers models fine-tuned on financial data).

- *Reasoning:* FinBERT understands that "bullish" and "positive" are synonyms in finance, whereas a general model might miss this nuance. It captures the semantic closeness of "Quantitative Easing" and "Bond Buying".[32]

5.1.2 Dimensionality Reduction (UMAP)

Financial news vectors are high-dimensional (768 dimensions for BERT). **UMAP** (Uniform

Manifold Approximation and Projection) is used to reduce this to a lower dimension (e.g., 5 components) to allow for density-based clustering.

- *Configuration:* n_neighbors=15 and n_components=5 are standard starting points. This preserves the local structure of the data (grouping similar articles) while maintaining global structure (keeping "FX" topics distinct from "Equities").[35]

5.1.3 Clustering (HDBSCAN)

HDBSCAN is a hierarchical density-based clustering algorithm. Unlike K-Means, it does not require specifying the number of clusters in advance, which is crucial because the number of daily news stories varies wildy.

- *Parameter Tuning:* min_cluster_size is set to 3. This ensures that even small but distinct stories (e.g., a specific bond issuance by a single issuer) are identified as a topic, rather than being treated as noise.[36]

5.1.4 Topic Representation (c-TF-IDF)

Once clusters are formed, BERTopic calculates the **Class-based TF-IDF** scores to identify the most representative words for each topic.

- *Result:* A cluster might be labeled 0_inflation_rate_hike_fed.
- *Enhancement:* The engine uses "KeyBERT-inspired" representation to refine these labels, ensuring they are semantically descriptive rather than just frequent words.[38]

---

6. Generative AI and Summarization: The Morning Brief

The extracted topics need to be synthesized into a coherent narrative. This is the "Morning Summary." The system employs abstractive summarization using Hugging Face Transformers.

6.1 Model Selection: Pegasus vs. BART

The choice of model is critical.

- **BART (facebook/bart-large-cnn):** excellent for general news but can be verbose.
- **Pegasus (google/pegasus-xsum or human-centered-summarization/financial-summarization-pegasus):** Designed specifically for abstractive summarization. The "financial-summarization-pegasus" model is fine-tuned on financial news, making it the superior choice for this engine. It is less likely to hallucinate numbers and more likely to capture the "bottom line" of a financial report.[40]

6.2 The Summarization Workflow

The summarization process faces a token limit challenge. A cluster might contain 20 articles, far exceeding the input limit of the model (usually 512 or 1024 tokens).

Strategy: Map-Reduce with LangChain

1. **Selection:** Identify the 3-5 most representative documents in the cluster (those closest to the cluster centroid in the embedding space).[38]
2. **Map:** Summarize each of these 3-5 articles individually using Pegasus.
3. **Reduce:** Concatenate the summaries and run a final summarization pass to produce a single, cohesive paragraph describing the topic.[42]

This ensures that the final summary captures the consensus view while highlighting key details from different sources.

---

7. Pipeline Orchestration: Apache Airflow

To run this complex workflow daily without manual intervention, **Apache Airflow** is used for orchestration. Airflow allows us to define the pipeline as a Directed Acyclic Graph (DAG), managing dependencies and retries.

7.1 DAG Architecture

The DAG (daily_financial_news) runs on a schedule (e.g., 0 6 * * * for 6 AM daily).

Tasks:

1. ingest_investing_com: Runs the Playwright scraper.
2. ingest_forexlive: Runs the RSS parser.
3. ingest_apis: Fetches rates from Alpha Vantage/FMP.
4. validate_data: Checks if article count > threshold. If not, alerts the admin (prevents processing empty data).
5. preprocess_nlp: Runs cleaning and spaCy extraction.
6. cluster_news: Runs BERTopic.
7. generate_summaries: Runs the Transformer models on clusters.
8. compile_report: Generates HTML.
9. send_email: Dispatches the report via SMTP.[44]

7.2 Docker Implementation

The entire system is containerized. A docker-compose.yaml file defines the services:

- postgres: Metadata database for Airflow.
- webserver: Airflow UI.
- scheduler: Airflow scheduler.
- worker: The customized Python environment containing playwright, spacy, and torch. Note that the worker image must be substantial to hold the ML models.[46]

---

8. Intelligence Delivery: Reporting and Visualization

The output of the engine is an HTML email report generated using **Jinja2** templates. This format is universally accessible and professional.

8.1 The "Morning Brief" Template

The Jinja2 template dynamically renders the data.

- **Market Snapshot Table:** A Markdown/HTML table at the top displaying the API-fetched rates (EUR/USD, 10Y Yield) with day-over-day changes.
- **Top Stories:** A loop iterates through the top 5 BERTopic clusters, displaying the generated summary and links to the representative articles.
- **Deal Watch Table:** A structured table displaying extracted deals.
  - *Columns:* Issuer | Instrument Type | Size | Maturity | Coupon (if detected).
  - *Source:* Populated by the EntityRuler extraction logic.
- **Regulatory Radar:** A section highlighting any articles flagged with regulatory keywords.[47]

8.2 Second-Order Insights

The report goes beyond aggregation by presenting "second-order" insights derived from the data structure:

- **Consensus Score:** Derived from the density of the BERTopic cluster. A high density implies high market consensus on a story.
- **Semantic Drift:** By comparing today's cluster centroids with yesterday's, the system can detect shifting narratives (e.g., a shift in discourse from "transitory inflation" to "entrenched inflation") before it becomes headline news.

---

9. Detailed Step-by-Step Implementation Guide

Phase 1: Environment Setup

1. Initialize Project: Use cookiecutter-data-science to create the directory structure.
   cookiecutter https://github.com/drivendata/cookiecutter-data-science
2. **Dependencies:** Create a requirements.txt with spacy, bertopic, playwright, transformers, apache-airflow.
3. Models: Download the necessary models.
   python -m spacy download en_core_web_trf
   playwright install

Phase 2: Building the Scraper

1. Create src/data/scraper.py.
2. Implement the InvestingScraper class using Playwright. Use page.wait_for_selector to ensure dynamic content loads.
3. Implement the ForexLiveScraper using feedparser.
4. Test scraping locally and save raw data to data/raw/.

Phase 3: NLP Pipeline Construction

1. Create src/features/preprocessing.py for regex cleaning.
2. Create src/features/ner.py. Load en_core_web_trf. Define the EntityRuler patterns for currencies, maturities, and the G-SIB issuer list.
3. Test extraction on a sample set of articles. Verify that "10-year" and "JPMorgan" are correctly tagged.

Phase 4: Clustering and Summarization

1. Create src/models/topic_model.py. Initialize BERTopic with FinBERT embeddings.
2. Implement the summarize_cluster function using the google/pegasus-xsum pipeline.
3. Run the full flow on the data/raw/ dataset to generate data/processed/daily_summary.json.

Phase 5: Automation

1. Define the Airflow DAG in dags/news_pipeline.py.
2. Set up the docker-compose.yaml for Airflow.
3. Mount the local source code into the Docker container.
4. Start the scheduler: docker-compose up.

Conclusion

The implementation of this Banking News and Deal Intelligence Engine provides a formidable competitive advantage. By automating the ingestion of over 100 daily articles and applying state-of-the-art NLP, the system distills the noise of the global markets into a clear, actionable signal. For the period of Feb 2025 – May 2025, capturing the nuances of regulatory shifts and credit cycle turns via this engine will empower deeper, faster decision-making in the FX and Rates trading desks. This roadmap provides the comprehensive technical foundation required to realize this capability.

Chapter 1: Strategic Architecture for Financial Intelligence

1.1 The Information Asymmetry Challenge

In the high-frequency world of institutional finance, information is the primary asset. However, the sheer volume of data produced daily—central bank speeches, regulatory filings, earnings calls, and geopolitical updates—far exceeds human processing capacity. A typical credit analyst might read 20-30 headlines a day; the market generates thousands.

The "Banking News and Deal Intelligence Engine" is designed to solve this asymmetry. It is not merely a news reader; it is a **Deal Intelligence System**. Its primary mandate is to identify actionable commercial opportunities ("Deals") and risk vectors ("Regulatory Events") hidden within the unstructured text of the web.

1.1.1 The "Deal" in News

What constitutes a "Deal" in this context?

1. **New Issuance:** A corporation or sovereign entity announcing a new bond sale.
    - *Data Points:* Issuer Name, Currency, Maturity, Yield/Coupon, Book Size.
2. **Credit Event:** A rating upgrade/downgrade or a default warning.
    - *Data Points:* Issuer, Rating Agency, Old Rating, New Rating.
3. **Regulatory Shift:** A change in capital requirements or a significant fine.
    - *Data Points:* Regulator (SEC, ECB), Target Bank, Fine Amount, Regulation ID (e.g., Basel IV).

The system architecture focuses on extracting these structured data points from unstructured narratives.

1.2 System Architecture and Tech Stack

The system is architected as a modular pipeline, ensuring that each component can be upgraded independently. The choice of Python is driven by its dominance in both Data Engineering (Airflow) and NLP (spaCy, Hugging Face).[3]

1.2.1 Core Components

- **Orchestrator: Apache Airflow**. It manages the workflow, handles retries, and provides a dashboard for monitoring pipeline health.[45]
- **Ingestion Layer: Playwright** (for dynamic scraping) and **BeautifulSoup** (for static parsing). **Alpha Vantage** and **EODHD** APIs provide the quantitative backbone.[6]
- **NLP Layer: spaCy** (with Transformers) for Entity Recognition. **BERTopic** for Clustering. **Hugging Face** (Pegasus) for Summarization.
- **Storage: PostgreSQL** for metadata and structured deal data. **S3** (or local MinIO) for raw HTML storage (Data Lake pattern).
- **Presentation: Jinja2** for HTML report generation.[48]

1.2.2 Project Structure (Cookiecutter Data Science)

To ensure the project is production-ready, we adopt the **Cookiecutter Data Science** directory structure. This standardizes the layout, making it easy for teams to collaborate.[4]

```
banking_intelligence_engine/
├── dags/ # Airflow Directed Acyclic Graphs
│ └── daily_news_pipeline.py
├── src/ # Source code for use in this project
│ ├── init.py
│ ├── data/ # Scripts to download or generate data
```

```
│ │ ├── scrapers.py # Playwright and BS4 logic
│ │ └── api_client.py # Alpha Vantage / FMP interactions
│ ├── features/ # Scripts to turn raw data into features
│ │ ├── preprocessing.py # Regex cleaning
│ │ └── ner_pipeline.py # spaCy EntityRuler definitions
│ ├── models/ # Scripts to train models and make predictions
│ │ ├── clustering.py # BERTopic configuration
│ │ └── summarization.py # Pegasus inference logic
│ └── visualization/ # Scripts to create exploratory and results oriented visualizations
│ └── email_generator.py # Jinja2 templates
├── data/ # Data directory
│ ├── raw/ # The original, immutable data dump
│ ├── interim/ # Intermediate data that has been transformed
│ └── processed/ # The final, canonical data sets for modeling
├── models/ # Trained and serialized models
├── requirements.txt # The requirements file for reproducing the analysis environment
├── Dockerfile # For containerizing the worker
└── docker-compose.yaml # Orchestrating Airflow services
```

This structure ensures that raw data is never overwritten (immutability) and that the modeling logic is separated from the execution logic.50

Chapter 2: Data Acquisition Framework

2.1 The Hybrid Acquisition Strategy

The "Feb 2025 – May 2025" timeframe requires a system that is robust against website changes. Relying solely on scraping is risky; relying solely on APIs is expensive and often misses niche news. A hybrid approach provides redundancy.

2.2 API Integration (The Quantitative Backbone)

We utilize APIs to fetch the "Market Snapshot" data—the quantitative context for the news.

2.2.1 Alpha Vantage & Financial Modeling Prep

- **Alpha Vantage:** Used for FX Rates and Economic Indicators (e.g., CPI, GDP releases). The FX_DAILY endpoint provides the OHLC data for major pairs.[6]
- **Financial Modeling Prep (FMP):** Used for "Stock News" and specific forex feeds. FMP's stock_news endpoint allows filtering by tickers, which is useful for tracking specific G-SIB banks.[7]

2.2.2 EODHD News API

EODHD is particularly valuable for its pre-tagged news API. It aggregates stories and provides a sentiment score. This serves as a validation layer: if our internal NLP model says a story is "negative," but EODHD says "positive," the system can flag this discrepancy for human review.[8]

2.3 The Web Scraping Architecture

For "Deal Intelligence," we need to go where the deals are discussed: specialized financial portals.

2.3.1 Target: Investing.com (Dynamic Content)

Investing.com is a treasure trove of bond market data, but it is heavily dynamic. A standard requests.get() call yields an empty skeleton. We must use **Playwright**.

Why Playwright?

Playwright (by Microsoft) is superior to Selenium for this task because:

1. **Speed:** It uses the DevTools protocol directly, making it faster.
2. **Resilience:** It handles "Shadow DOM" and modern React/Angular apps natively.
3. **Headless:** It runs efficiently in a Docker container.[13]

Implementation Detail:

The scraper must navigate to the "Fixed Income" news section. It waits for the specific div containing the news feed to appear using page.wait_for_selector(). It then extracts the href (link) and title attributes. Crucially, it must handle pagination or "infinite scroll" if historical data is needed, though for a daily engine, the first page usually suffices.52

2.3.2 Target: ForexLive (RSS Feeds)

ForexLive provides excellent real-time commentary. They publish RSS feeds which are far easier to consume than scraping HTML.

- **Tool:** feedparser library.
- **Advantage:** Low bandwidth, no JavaScript execution required, highly reliable structure.
- **Process:** The engine polls the RSS feed every hour. It checks the published_parsed timestamp to only ingest articles published since the last run.[10]

2.4 Anti-Bot Evasion and Reliability

To ensure the scraper runs daily without being blocked:

1. **User-Agent Rotation:** We use the fake-useragent library to generate a new UA string for every session.
2. **Request Throttling:** We never hit a site with concurrent requests. We serialize requests and add a random delay (2-5 seconds) between them.
3. **Proxy Management:** In a production environment, routing requests through a rotating proxy service (like Bright Data or ScraperAPI) is recommended to mask the scraper's IP address.[16]

Chapter 3: Financial Natural Language Processing

3.1 The Preprocessing Pipeline

Once the raw HTML/XML is ingested, it is converted to text. This text is "noisy." Financial NLP requires a specific cleaning pipeline.

3.1.1 Regex-Based Cleaning

The src/features/preprocessing.py module contains a suite of regex functions.

- **Strip HTML:** Remove <br>, , etc. (if not handled by BeautifulSoup).
- **Remove Disclaimers:** Financial news often ends with "This is not investment advice..." This boilerplate skews clustering and must be removed.
- **Whitespace Normalization:** Convert non-breaking spaces ( ) and multiple newlines into single spaces.

3.2 Financial Text Normalization

Standard NLP models struggle with financial notation. We normalize these *before* passing text to spaCy.

A. Ticker/Currency Normalization:

Traders use slang. The pipeline maps synonyms to standard ISO codes.

- "Looney" / "Loonie" -> CAD
- "Cable" -> GBP/USD
- "Fiber" -> EUR/USD
- "Aussie" -> AUD
- "Kiwi" -> NZD
  This ensures that "The Loonie rallied" and "CAD strengthened" are treated as the same topic.25

B. Number Normalization:

"1.2 billion" and "$1.2bn" and "1,200,000,000" are the same. We normalize all large numbers to a standard scientific notation or a simplified string format (e.g., 1.2B) to help the model recognize deal sizes.

3.3 Sentiment Dictionary Integration

While transformer-based sentiment analysis is powerful, dictionary-based methods provide a stable baseline. We integrate the **Loughran-McDonald Master Dictionary**, the academic standard for financial sentiment.

- **Mechanism:** The dictionary tags words like "restructuring" or "impairment" as *Negative*, whereas a general dictionary might see them as neutral.
- **Application:** For every article, we calculate a LM_Sentiment_Score. This score helps

prioritize "negative" news (often risks) in the morning summary.[20]

Chapter 4: Advanced Entity Recognition (NER)

4.1 The Core Challenge: Financial Named Entities

The request asks for extraction of **Currencies**, **Issuers**, **Maturities**, and **Regulatory Events**. A standard en_core_web_sm model fails here:

- It tags "Bank of America" as ORG (Good).
- It tags "10-year" as DATE (Too generic).
- It tags "EUR/USD" as ORG or fails to tag it (Bad).

4.2 The Solution: Hybrid NER with spaCy

We use **spaCy v3** with a transformer backbone (en_core_web_trf) for accuracy, augmented by a rule-based **EntityRuler** for precision.

4.3 Implementing the EntityRuler

The EntityRuler is a pipeline component that looks for exact matches or regex patterns. We add this *before* the main NER component to ensure our domain-specific rules take precedence.[22]

4.3.1 Maturity Extraction (The "Tenor")

In Rates markets, the maturity of the bond is the product.
Regex Pattern: r"(\d+)-(year|yr|Y|month|mo|M)"
This captures "10-year", "30Y", "6M".
Implementation:

Python

```
patterns =},
   {"label": "MATURITY", "pattern":}}]}
]
```

Post-Processing: We verify that these entities modify specific nouns (note, bond, yield) to avoid capturing "10-year anniversary".[2]

4.3.2 Currency Pair Extraction

Regex Pattern: r"^[A-Z]{3}(/|-|)[A-Z]{3}$"
This captures "EUR/USD", "EURUSD", "EUR-USD".
Implementation:

Python

```
patterns.append(
   {"label": "CURRENCY_PAIR", "pattern":{3}/?[A-Z]{3}$"}}]}
)
```

This allows the system to extract the specific pair being discussed, which is vital for the "Market Snapshot" table.[24]

4.3.3 Issuer Identification (The G-SIB List)

To accurately identify banks, we import the official FSB G-SIB List.
List: JP Morgan, Citigroup, HSBC, BNP Paribas, Deutsche Bank, etc..26
Implementation: We generate a pattern for every bank on this list and tag them as ISSUER.
This differentiates a "Bank" (the institution) from a generic "bank" (the building).

Python

```
g_sibs =
for bank in g_sibs:
   patterns.append({"label": "ISSUER", "pattern":})
```

4.3.4 Regulatory Event Detection

This is a high-value requirement. We build a taxonomy of triggers.
Keywords: AML, KYC, Sanctions, Fine, Penalty, Enforcement, Probe, Investigation, Basel III, Capital Buffer.28
Logic: If an article contains a REGULATORY_KEYWORD and an ISSUER, it is flagged as a REGULATORY_EVENT.

- *Example:* "FCA fines Barclays" -> Regulator: FCA, Trigger: fines, Target: Barclays.
- *Value:* This allows the report to highlight compliance risks immediately.[29]

Chapter 5: Semantic Clustering with BERTopic

5.1 Why Clustering?

With 100+ articles, simply listing them is useless. We need to group them into "stories." **BERTopic** is the state-of-the-art for this. It uses embeddings to find clusters of documents that are semantically similar, even if they don't share the exact same keywords.[32]

5.2 The BERTopic Pipeline for Finance

5.2.1 Step 1: Embeddings (FinBERT)

BERTopic defaults to all-MiniLM-L6-v2. However, for financial text, context is everything. We use **FinBERT** (ProsusAI/finbert), a BERT model pre-trained on financial sentiment datasets.

- *Benefit:* It generates vectors where "inflation" and "CPI" are close, and "default" and "bankruptcy" are close.[34]
- *Code snippet:*
  Python
  ```
  from bertopic import BERTopic
  from sentence_transformers import SentenceTransformer

  # Load FinBERT
  embedding_model = SentenceTransformer("ProsusAI/finbert")
  ```

5.2.2 Step 2: Dimensionality Reduction (UMAP)

We use UMAP to reduce the 768-dimensional FinBERT vectors to 5 dimensions.

- *Config:* n_neighbors=15, n_components=5, metric='cosine'.
- *Goal:* To prepare the data for density clustering while preserving the "neighborhoods" of related news.[35]

5.2.3 Step 3: Clustering (HDBSCAN)

HDBSCAN finds the clusters.

- *Config:* min_cluster_size=3. This is crucial. In a daily feed of 100 articles, a "topic" might only consist of 3 articles (e.g., a specific merger). Setting this too high (e.g., 20) would erase these niche but valuable signals.[36]

5.2.4 Step 4: Topic Representation (c-TF-IDF)

We need to label the clusters. BERTopic uses c-TF-IDF to find words that are unique to the cluster.

- *Refinement:* We use KeyBERTInspired representation. This ensures the keywords are not just frequent (like "said" or "today") but semantically related to the topic (like "merger" or "yield").[38]

Chapter 6: Generative AI and Summarization

6.1 Abstractive Summarization

Once we have a cluster of 10 articles about "Fed Rate Hike," we need one paragraph summarizing them. We use **Abstractive Summarization** (generating new sentences) rather than **Extractive** (copy-pasting sentences).

6.2 Model Selection: Pegasus vs. BART

- **Pegasus (Google):** Pre-trained with "Gap Sentence Generation." It excels at synthesizing information. We use the human-centered-summarization/financial-summarization-pegasus checkpoint, fine-tuned on financial news.[40]
- **BART (Facebook):** Good, but often more verbose. For a "Morning Brief," brevity is key. Pegasus is preferred.[41]

6.3 Handling Long Contexts (The Map-Reduce Pattern)

A cluster might have 5,000 words of text. Pegasus has a limit (e.g., 512 or 1024 tokens). We cannot feed it all.
Solution:

1. **Select Representatives:** BERTopic identifies the most representative documents in a cluster. We pick the top 3.
2. **Map:** Summarize each of the 3 documents individually.
3. **Reduce:** Concatenate the 3 summaries and summarize them again.

- *LangChain:* This pattern is easily implemented using LangChain's load_summarize_chain with chain_type="map_reduce", but can also be coded manually with Hugging Face pipelines for finer control.[42]

Chapter 7: Pipeline Orchestration and Automation

7.1 Apache Airflow

This engine must run every morning at 6 AM. **Apache Airflow** is the industry standard for this.

7.2 The DAG Structure (daily_financial_news.py)

The pipeline is defined as a graph.

1. **Start**
2. **Parallel Tasks:** scrape_investing, scrape_forexlive, fetch_api_data.
3. **Sensor:** Wait for all ingestion to finish.
4. **Task:** preprocess_data (Cleaning).
5. **Task:** run_ner (spaCy extraction).
6. **Task:** run_bertopic (Clustering).
7. **Task:** summarize_topics (Pegasus).
8. **Task:** generate_report (Jinja2).
9. **Task:** email_report (SMTP).
10. **End**.[44]

7.3 Docker Containerization

To ensure the environment (with complex libraries like Playwright and PyTorch) is reproducible, we use Docker.
Dockerfile:

Dockerfile

```
FROM python:3.9-slim
RUN apt-get update && apt-get install -y libpq-dev gcc
COPY requirements.txt.
RUN pip install -r requirements.txt
RUN python -m spacy download en_core_web_trf
RUN playwright install
RUN playwright install-deps
COPY. /app
WORKDIR /app
```

Docker Compose: Sets up the Airflow Webserver, Scheduler, and Postgres DB. This makes the entire engine portable.[46]

Chapter 8: Intelligence Delivery and Reporting

8.1 The User Interface: Email

The primary interface for this system is the email inbox. The report must be clean, readable on mobile, and data-rich.

8.2 Jinja2 Templating

We use **Jinja2** to generate HTML.

- **Dynamic Tables:** We iterate over the extracted data to build tables.
  HTML
  ```
  <table>
    <thead><tr><th>Issuer</th><th>Maturity</th><th>Type</th></tr></thead>
    <tbody>
    {% for deal in deals %}
      <tr>
        <td>{{ deal.issuer }}</td>
        <td>{{ deal.maturity }}</td>
        <td>{{ deal.type }}</td>
      </tr>
    {% endfor %}
    </tbody>
  </table>
  ```

.[48]

8.3 Report Sections

1. **Market Snapshot:** Quantitative context (rates from API).
2. **Executive Summary:** A 3-sentence overview of the biggest story (Cluster #0).
3. **Top Themes:** The top 5 clusters, summarized.
4. **Deal Watch:** The table of extracted Issuers/Maturities.
5. **Regulatory Radar:** The list of compliance-related articles.

Chapter 9: Future Outlook and Advanced Analytics

9.1 Semantic Drift Analysis

By tracking the centroids of topics over time (using BERTopic's dynamic topic modeling), the system can detect "Semantic Drift."

- *Example:* A topic about "Housing Market" might slowly drift towards "Subprime" keywords. Detecting this vector shift *before* the market prices it in is a source of "Alpha" (trading advantage).

9.2 The "Feb 2025" Context

In the specified timeframe, markets are expected to be sensitive to central bank divergence. The engine's ability to separate "ECB" news from "Fed" news (via NER) and cluster them separately allows for a clear view of this divergence, providing users with a distinct strategic edge.

Note: This report utilizes research snippets [6] through [51] to substantiate the architectural and technical decisions detailed above.

Works cited

1. Machine Learning and Data Science Blueprints for Finance - School of Computing e-Library, accessed on December 13, 2025, https://soclibrary.futa.edu.ng/books/Machine%20Learning%20and%20Data%20Science%20Blueprints%20for%20Finance%20(Hariom%20Tatsat,%20Sahil%20Puri,%20Brad%20Lookabaugh)%20(Z-Library).pdf
2. 6 – IS MUNI, accessed on December 13, 2025, https://is.muni.cz/th/l3y56/sample_data/frequency_lists/en_lingeaAll_cs_5m.fwl?stahnout=1;dk=FJzf_8QF
3. Python Data Pipeline: Frameworks & Building Processes - lakeFS, accessed on December 13, 2025, https://lakefs.io/blog/python-data-pipeline/
4. Using CookieCutter for Data Science Project Templates - ProjectPro, accessed on December 13, 2025, https://www.projectpro.io/article/cookiecutter-data-science/982
5. drivendataorg/cookiecutter-data-science: A logical, reasonably standardized, but flexible project structure for doing and sharing data science work. - GitHub, accessed on December 13, 2025, https://github.com/drivendataorg/cookiecutter-data-science
6. Alpha Vantage: Free Stock APIs in JSON & Excel, accessed on December 13, 2025, https://www.alphavantage.co/
7. Free Stock Market API and Financial Statements API... | FMP, accessed on December 13, 2025, https://site.financialmodelingprep.com/developer/docs
8. Financial News Feed and Stock News Sentiment data API | EODHD APIs Documentation, accessed on December 13, 2025, https://eodhd.com/financial-apis/stock-market-financial-news-api
9. G-Glitch404/investing.com-scraper: scrape news articles from investing.com as much as you want - GitHub, accessed on December 13, 2025, https://github.com/G-Glitch404/investing.com-scraper
10. RSS feeds - FX Markets, accessed on December 13, 2025, https://www.fx-markets.com/static/rss-feeds
11. Where do you get your news feed : r/Forex - Reddit, accessed on December 13, 2025, https://www.reddit.com/r/Forex/comments/7oudlk/where_do_you_get_your_news

_feed/
12. Web Scraping Financial News Using Python - Tutorials Point, accessed on December 13, 2025, https://www.tutorialspoint.com/web-scraping-financial-news-using-python
13. Playwright for Python Web Scraping Tutorial with Examples - ScrapingBee, accessed on December 13, 2025, https://www.scrapingbee.com/blog/playwright-for-python-web-scraping/
14. Python Playwright Web Scraping (Complete Guide) - Rayobyte, accessed on December 13, 2025, https://rayobyte.com/blog/playwright-web-scraping/
15. Web Scraping Financial News Using Python - GeeksforGeeks, accessed on December 13, 2025, https://www.geeksforgeeks.org/python/web-scraping-financial-news-using-python/
16. Playwright Web Scraping: Complete Guide for 2025 - ScraperAPI, accessed on December 13, 2025, https://www.scraperapi.com/web-scraping/playwright/
17. Web Scraping VS Official APIs for Financial Data: Stocks, Crypto, and Forex - Medium, accessed on December 13, 2025, https://medium.com/@trading.dude/web-scraping-vs-official-apis-for-financial-data-stocks-crypto-and-forex-89efb2a1818f
18. Regular Expression HOWTO — Python 3.14.2 documentation, accessed on December 13, 2025, https://docs.python.org/3/howto/regex.html
19. Regex : Data Extraction using Python, Pattern Detection for files. Fundamental Overview | by Chinmay Kapoor | Medium, accessed on December 13, 2025, https://medium.com/@kapoorchinmay231/regex-data-extraction-using-python-pattern-detection-for-files-fundamental-overview-e0f1342ddc9c
20. Loughran-McDonald sentiment lexicon — lexicon_loughran • textdata, accessed on December 13, 2025, https://emilhvitfeldt.github.io/textdata/reference/lexicon_loughran.html
21. Textual Analysis in Finance - Annual Reviews, accessed on December 13, 2025, https://www.annualreviews.org/doi/pdf/10.1146/annurev-financial-012820-032249
22. Linguistic Features · spaCy Usage Documentation, accessed on December 13, 2025, https://spacy.io/usage/linguistic-features
23. 5. Using SpaCy's EntityRuler, accessed on December 13, 2025, https://spacy.pythonhumanities.com/02_01_entityruler.html
24. ORE User Guide - Open Source Risk Engine, accessed on December 13, 2025, https://www.opensourcerisk.org/wp-content/uploads/2022/09/userguide.pdf
25. How can get ' USDJPY'(currency rates) with pandas and yahoo finance? - Stack Overflow, accessed on December 13, 2025, https://stackoverflow.com/questions/21483959/how-can-get-usdjpycurrency-rates-with-pandas-and-yahoo-finance
26. 2023 List of Global Systemically Important Banks (G-SIBs) - Financial Stability Board, accessed on December 13, 2025, https://www.fsb.org/2023/11/2023-list-of-global-systemically-important-banks-g-sibs/
27. 2024 List of Global Systemically Important Banks (G-SIBs) - Financial Stability

Board, accessed on December 13, 2025, https://www.fsb.org/2024/11/2024-list-of-global-systemically-important-banks-g-sibs/
28. 41 key compliance terms you should know in 2023 | NorthRow, accessed on December 13, 2025, https://www.northrow.com/blog/41-key-compliance-terms-you-should-know-in-2023
29. Glossary of Financial Crime Compliance / AI Terms - Silent Eight, accessed on December 13, 2025, https://www.silenteight.com/explore-learn/glossary-of-financial-crime-compliance-ai-terms
30. Four ways to apply NLP in financial services - LSEG, accessed on December 13, 2025, https://www.lseg.com/en/insights/data-analytics/four-ways-to-apply-nlp-in-financial-services
31. Leveraging natural language processing for automated regulatory compliance in financial reporting - Global Journal of Engineering and Technology Advances, accessed on December 13, 2025, https://gjeta.com/sites/default/files/GJETA-2025-0187.pdf
32. A Practical Guide to BERTopic for Transformer-Based Topic Modeling, accessed on December 13, 2025, https://towardsdatascience.com/a-practical-guide-to-bertopic-for-transformer-based-topic-modeling/
33. NLP Tutorial: Topic Modeling in Python with BerTopic | by Davis David | Geek Culture, accessed on December 13, 2025, https://medium.com/geekculture/nlp-tutorial-topic-modeling-in-python-with-bertopic-da760e1d03aa
34. ProsusAI/finbert - Hugging Face, accessed on December 13, 2025, https://huggingface.co/ProsusAI/finbert
35. Accelerating Topic modeling with RAPIDS and BERT models | by Vibhu Jawa - Medium, accessed on December 13, 2025, https://medium.com/rapids-ai/accelerating-topic-modeling-with-rapids-and-bert-models-be9909eeed2
36. 3. Clustering - BERTopic - Maarten Grootendorst, accessed on December 13, 2025, https://maartengr.github.io/BERTopic/getting_started/clustering/clustering.html
37. The Algorithm - BERTopic, accessed on December 13, 2025, https://maartengr.github.io/BERTopic/algorithm/algorithm.html
38. 6A. Representation Models - BERTopic - Maarten Grootendorst, accessed on December 13, 2025, https://maartengr.github.io/BERTopic/getting_started/representation/representation.html
39. Update Topic Representations - BERTopic - Maarten Grootendorst, accessed on December 13, 2025, https://maartengr.github.io/BERTopic/getting_started/topicrepresentation/topicre

presentation.html
40. human-centered-summarization/financial-summarization-pegasus - Hugging Face, accessed on December 13, 2025, https://huggingface.co/human-centered-summarization/financial-summarization-pegasus
41. Summarizing Business News: Evaluating BART, T5, and PEGASUS for Effective Information Extraction - IIETA, accessed on December 13, 2025, https://www.iieta.org/download/file/fid/132319
42. 6B. LLM & Generative AI - BERTopic - Maarten Grootendorst, accessed on December 13, 2025, https://maartengr.github.io/BERTopic/getting_started/representation/llm.html
43. Clustering-Based Text Summarization Using LangChain's Map-Reduce Method - Medium, accessed on December 13, 2025, https://medium.com/@ankita.bagaria8/clustering-based-text-summarization-using-langchains-map-reduce-method-5f2c27db5fee
44. anastasiia-p/airflow-ml: Airflow Pipeline for Machine Learning - GitHub, accessed on December 13, 2025, https://github.com/anastasiia-p/airflow-ml
45. Building a Simple Data Pipeline - Apache Airflow, accessed on December 13, 2025, https://airflow.apache.org/docs/apache-airflow/stable/tutorial/pipeline.html
46. NLP Pipeline: Key Steps to Process Text Data | Airbyte, accessed on December 13, 2025, https://airbyte.com/data-engineering-resources/natural-language-processing-pipeline
47. Create an Email Alert System with Python to get Earnings Data - InsightBig, accessed on December 13, 2025, https://www.insightbig.com/post/create-an-email-alert-system-with-python-to-get-earnings-data
48. Send an HTML Email Template with Python and Jinja2 - DEV Community, accessed on December 13, 2025, https://dev.to/carola99/send-an-html-email-template-with-python-and-jinja2-1hd0
49. Building an ETL Pipeline with Airflow - DataCamp, accessed on December 13, 2025, https://www.datacamp.com/tutorial/building-an-etl-pipeline-with-airflow
50. What is the best project structure for a Python application? [closed] - Stack Overflow, accessed on December 13, 2025, https://stackoverflow.com/questions/193161/what-is-the-best-project-structure-for-a-python-application
51. NLP Project Pipeline Structure Accessible with One Click. | by Mayur - AI Engineer | Medium, accessed on December 13, 2025, https://medium.com/@mayurpatil-in/nlp-project-template-pipeline-d88cf9df3525
52. How to Scrape Forex Markets Using Beautiful Soup - ScraperAPI, accessed on December 13, 2025, https://www.scraperapi.com/blog/how-to-scrape-forex-markets-using-beautiful-soup/
53. Scraping historical data from investing.com - python - Stack Overflow, accessed

on December 13, 2025, https://stackoverflow.com/questions/77963587/scraping-historical-data-from-investing-com
54. SEC Filings Dictionary-Based Sentiment Analysis - WRDS, accessed on December 13, 2025, https://wrds-www.wharton.upenn.edu/pages/classroom/sec-filings-dictionary-based-sentiment-analysis/